

Comparison of Machine Learning Algorithms: Cost-Sensitive Evaluation and Multi-class Reduction Strategies

Project Plan

Muhammad Subhan Ahmad

Final Year Individual Project

Department of Computer Science

Royal Holloway, University of London

1 Abstract

Machine learning lets a program learn how to solve a class of problems from a training set, rather than being given an explicit solution by hand. Many families of algorithms exist for this, from simple methods such as nearest neighbors to more sophisticated ones such as support vector machines, and each family makes very different assumptions about the data it is given. Nearest neighbor methods assume that points which are close together tend to share a label, decision trees carve the feature space up into regions using a sequence of simple rules, and support vector machines look for the boundary that separates the classes by the widest possible margin, optionally after mapping the data into a richer feature space through a kernel. When these algorithms are used through a library, the interesting design decisions behind them, the distance metric, the tie-breaking rule, the impurity measure, the optimization method, are hidden away. I want to open that box.

This project implements these algorithms from first principles in Java, checks them against an established toolkit to make sure they are correct, and then compares their accuracy, robustness and computational cost on well-known benchmark data sets from the UCI and Delve repositories. The comparison itself will be done properly, using a held-out test set, k-fold cross-validation and ROC analysis rather than a single accuracy number, and preprocessing (particularly normalization of the attributes) will be studied as part of the experiment rather than treated as a box to tick before the real work starts.

What I think will make this project stand out is the extension. Almost every classifier is trained to minimise overall error, which quietly assumes that every mistake costs the same. That is rarely true. Letting one spam email through is a minor annoyance; blocking a genuine, important email is not. Approving a loan for a customer who later defaults costs a bank far more than turning away a customer who would have paid it back. My main extension studies how each of the implemented algorithms can be adapted to this kind of asymmetric cost, and whether the ranking of algorithms changes once they are judged by expected cost and ROC analysis instead of plain accuracy. Because a multi-class support vector machine only really exists once binary classifiers have been combined, I will also run a smaller secondary study comparing the standard ways of doing that combination and how each one scales as the number of classes grows.

The resulting software will be built around a proper object-oriented design and a modern software engineering workflow, version control, automated testing and a documented design, and the final report

will cover the theory behind each algorithm, the decisions made while implementing it, the engineering process used to build the software, and the experiments and conclusions that come out of it.

2 Choice of Programming Language

The specification allows Java or C++, and I considered Python as well before deciding. The table below summarizes why I have chosen Java for this project.

Language	Strengths for this project	Why not chosen
Java	Strong, statically checked object-oriented design fits the required Classifier hierarchy naturally. JUnit gives mature unit testing. Automatic memory management removes a whole class of bugs that would otherwise slow down development. Swing or JavaFX gives a straightforward path to the optional GUI. Performance is more than adequate for data sets of this size once basic numerical code is written sensibly.	Chosen.
C++	The fastest of the three options, which would help if the data sets were very large or the SVM optimization needed to be extremely tight.	Manual memory management and a weaker standard testing ecosystem add engineering overhead that is not needed here and would eat into time better spent on the algorithms themselves.
Python	Very fast to prototype in, and the natural home of NumPy and scikit-learn, which is useful for checking results.	Encourages leaning on library calls rather than writing the algorithms from scratch, and a typical from-scratch implementation is noticeably slower than the same algorithm in Java or C++, which matters once I am timing training and prediction as part of the comparison.

Java gives the best balance for what this project needs: a clean object-oriented design that the specification asks for, dependable testing, and enough performance to run a proper set of experiments, without the extra engineering overhead of C++ or the temptation to quietly fall back on library implementations that Python invites. scikit-learn and Weka will still be used, but only afterwards, to check that my own implementations are producing correct results.

3 Core Research Questions

- RQ1: How much do different k-NN tie-breaking strategies change accuracy, and does that depend on whether k is small or even?
- RQ2: Do the different decision tree impurity measures (Gini index, entropy, misclassification error) lead to meaningfully different trees and accuracy, or mostly agree with each other?
- RQ3: How much difference does preprocessing make, normalizing attributes against normalizing examples, and does the answer change from one algorithm to the next?
- RQ4: How do the algorithm families rank against each other on accuracy, training time and prediction time, across data sets of different size and dimensionality?

4 Extension: Asymmetric Errors and ROC Analysis

Working title: When Mistakes Are Not Equal: A Comparison of Cost-Sensitive Nearest Neighbors, Decision Trees and Support Vector Machines for Spam and Credit Risk Classification.

This extension asks how each core algorithm can be adapted once mistakes are allowed to cost different amounts, whether by shifting the decision threshold, by weighting training examples, or through an algorithm-specific change such as cost-weighted impurity in a tree or a class-specific penalty in an SVM. It also asks whether the ranking of algorithms changes once they are judged by expected cost and ROC analysis rather than accuracy. I will run it on the Spambase and Statlog German Credit data sets, with Breast Cancer Wisconsin (Diagnostic) giving a medical diagnosis angle where missing a genuine case is clearly worse than a false alarm.

Secondary study: multi-class reduction strategies. A support vector machine is naturally a binary classifier, so a multi-class problem has to be reduced to a set of binary ones. As a smaller, secondary study I will compare one-against-the-rest, one-against-one and error-correcting output codes for accuracy and for how training and prediction time scale as the number of classes grows, using the Optical Digits and Statlog Landsat data sets.

5 Timeline

Term one covers implementation and the early deliverables. Term two covers the harder algorithms, the extension, the experiments and the final report. Week numbers should be lined up with the actual term dates once they are confirmed.

5.1 Term 1

Week 1: Background reading on machine learning fundamentals (Mitchell; Hastie, Tibshirani and Friedman). Set up the project structure in the GitLab repository (code, documents, diary and AI declaration) and the Overleaf report. Sketch the Classifier interface that every algorithm will implement.

Week 2: Implement 1-NN and k-NN with Euclidean and Manhattan distance and several tie-breaking strategies. Generate simple artificial data (Gaussian blobs, XOR, concentric circles) for early testing.

Week 3 to 4: Implement decision trees supporting Gini, entropy and misclassification error as impurity measures. Write JUnit tests for both k-NN and the tree on the artificial data.

Week 5: Write the two early deliverable report sections, k-NN with tie-breaking strategies, and decision trees with the different uniformity measures.

Week 6 to 7: Build the evaluation framework, hold-out split, k-fold cross-validation, ROC curves and AUC, confusion matrix, and the preprocessing module (attribute and example normalisation). Load Iris, Wine, Breast Cancer Wisconsin and Ionosphere from UCI.

Week 8: Run the first experiments for RQ1 to RQ3 and check the results against Weka on the same data.

Week 9: Start kernel k-NN, using linear, polynomial and RBF kernels.

Week 10 to 11: Interim report and presentation, with a little buffer time left over.

5.2 Term 2

Week 1 to 3: Implement a soft-margin SVM with kernels and an SMO-style optimiser, then extend it to multiple classes using one-against-one as the default.

Week 4: Full RQ4 experiments comparing every algorithm family on accuracy, training time and prediction time across all the data sets.

Week 5 to 6: Extension work, cost-sensitive versions of k-NN, the tree and the SVM, tested on Spambase, Statlog German Credit and Breast Cancer Wisconsin under an explicit cost matrix.

Week 7: Secondary study, one-against-the-rest and error-correcting output codes alongside one-against-one, compared on Optical Digits and Statlog Landsat.

Week 8: A small JavaFX GUI to visualise decision boundaries and ROC curves, if time allows.

Week 9: Final experiments, analysis and write-up of the findings for every research question.

Week 10 to 11: Final report, the software engineering write-up, and preparation for the viva, with buffer time at the end.

6 Risks and Mitigations

6.1 Hardware Failure

My main hardware could fail partway through the project, taking progress with it. I will keep all code under version control on the university GitLab and commit regularly, and keep the report under version control on Overleaf, with a copy of each report version in the documents folder of the repository, so everything that matters lives in the cloud and does not depend on one machine.

6.2 Poor Estimation of Tasks

It is easy to underestimate how long a task will take, and that can quietly eat into later weeks. I will break each task down into smaller pieces, research anything unfamiliar before committing to a time estimate, and check progress against the timeline every week rather than only at the milestones.

6.3 Uneven Balance Between Report and Code

Code and report can easily pull against each other, with one getting neglected while I focus on the other. I will write each report section as soon as the matching algorithm is implemented and tested, instead of saving all the writing for the end, and I have left buffer weeks at the end of each term for exactly this reason.

6.4 Implementation Errors That Quietly Distort the Results

A subtle bug in a from-scratch implementation could go unnoticed and undermine the whole comparison without me realising. To guard against this I will use test-driven development, check every algorithm on simple artificial data where the correct answer is already known, and cross-check results against Weka on the same data.

6.5 Flawed Experimental Design

Mistakes such as normalising data before splitting it, or tuning parameters on the test set, would quietly invalidate the comparison even if the code itself is correct. I will always fit preprocessing inside each cross-validation fold, keep test data strictly separate from training and validation data, and talk the experimental design through with my supervisor before running the main experiments.

6.6 Difficulty of the SVM Implementation

The support vector machine, especially its optimisation and its kernel extension, is the hardest part of the core work. I will start it early, in the first week of term two, work from Platt's original SMO paper, and build a simplified version first if the full optimiser turns out to take longer than expected.

6.7 Scope Creep From the Extension

The cost-sensitive extension and the secondary multi-class study both have room to grow well beyond the time available. The core algorithms and their evaluation are the non-negotiable deliverable, and the secondary study and the GUI are the first things I will drop if time runs short.

6.8 Large or Imbalanced Data Sets

Some of the extension data sets, Letter Recognition and the optional Credit Card Fraud set in particular, are either large or heavily imbalanced, which can make training slow or results unstable. I will profile the code early, start with smaller subsamples, and only move on to the full-sized data once the pipeline is already working correctly. Large data files will be kept outside the repository, on university online storage.

7 Software Engineering Approach

- A proper object-oriented design in Java, with a common Classifier interface so every algorithm can be trained, tested and compared through the same API.
- Test-driven development, with JUnit tests for each algorithm as it is written.
- Git and the university GitLab for version control, with regular, meaningful commits (at least five a week, each one small) rather than occasional large ones.
- The algorithms themselves are written by hand rather than taken from a library. Weka and scikit-learn are used only afterwards, to check that my results are correct.
- Design documentation, class diagrams and a short record of the module structure, kept up to date for the software engineering section of the final report.
- Any use of AI tools will be declared in an AI.md file at the top of the repository, and progress will be logged weekly in diary.md, newest entry first.

Acronyms

Term	Meaning
AUC	Area Under the (ROC) Curve.
ECOC	Error-Correcting Output Codes.
GUI	Graphical User Interface.
ML	Machine Learning.
OvO	One-against-One.
OvR	One-against-the-Rest.
ROC	Receiver Operating Characteristic.
SE	Software Engineering.
SMO	Sequential Minimal Optimization.
SVM	Support Vector Machine.
TDD	Test-Driven Development.

Glossary

Term	Meaning
GitLab	A cloud-hosted extension of the Git version control system with a web interface, provided by the university and used for the project's code, diary and documents.
Overleaf	An online LaTeX editor providing history, version control and collaboration.
Weka	A Java machine learning toolkit from the University of Waikato, used here to check my own implementations.
Kernel	A function that implicitly computes an inner product in a higher-dimensional feature space, letting a linear method such as an SVM or k-NN learn non-linear boundaries.

References

- [1] T. Hastie, R. Tibshirani and J. Friedman. The Elements of Statistical Learning, 2nd ed. Springer, New York, 2009.
- [2] T. M. Mitchell. Machine Learning. McGraw-Hill, New York, 1997.
- [3] V. N. Vapnik. The Nature of Statistical Learning Theory, 2nd ed. Springer, New York, 2000.
- [4] V. N. Vapnik. Statistical Learning Theory. Wiley, New York, 1998.
- [5] UCI Machine Learning Repository. <https://archive.ics.uci.edu>
- [6] Delve archive, University of Toronto. <https://www.cs.toronto.edu/~delve/>
- [7] T. Fawcett. An introduction to ROC analysis. Pattern Recognition Letters 27(8), 2006.
- [8] C. Elkan. The foundations of cost-sensitive learning. IJCAI, 2001.
- [9] C.-W. Hsu and C.-J. Lin. A comparison of methods for multiclass support vector machines. IEEE Trans. Neural Networks 13(2), 2002.
- [10] T. G. Dietterich and G. Bakiri. Solving multiclass learning problems via error-correcting output codes. JAIR 2, 1995.
- [11] J. Platt. Sequential minimal optimization: a fast algorithm for training support vector machines. Microsoft Research Tech. Report, 1998.
- [12] Weka (Java ML toolkit), University of Waikato. <https://ml.cms.waikato.ac.nz/weka/>